

REGEX

Nazanin Foroutan

AP Spring 2026 – Dr. Vahidi-Asl

Regular Expressions

- What is regex?
- Simple Match (literal text)
 - Example: cat matches "cat" in "catalog"
 - Case sensitivity note

Anchors & Position Matching

^
Start of
line/string

Example: ^Hello matches "
Hello world" but not "Say
Hello"

\$ end of
line/string

Example: world\$ matches
"Hello world"

Combined

^Start\$ matches exact
string "Start"

Character Classes & Negation

01 `[aeiou]` matches any vowel

02 `[0-9]` matches any digit

03 Negation: `[^aeiou]` matches anything except vowels

04 Example: `gr[ae]y` matches "gray" or "grey"

The Wildcard & Quantifiers

.

Dot

Any single
Character
Except newline

+

plus

One or more

*

star

Zero or more

?

Question mark

Zero or one

{3}

Exactly 3 times

{2, 5}

Between 2 and 5
times

`\d`

Any decimal digits

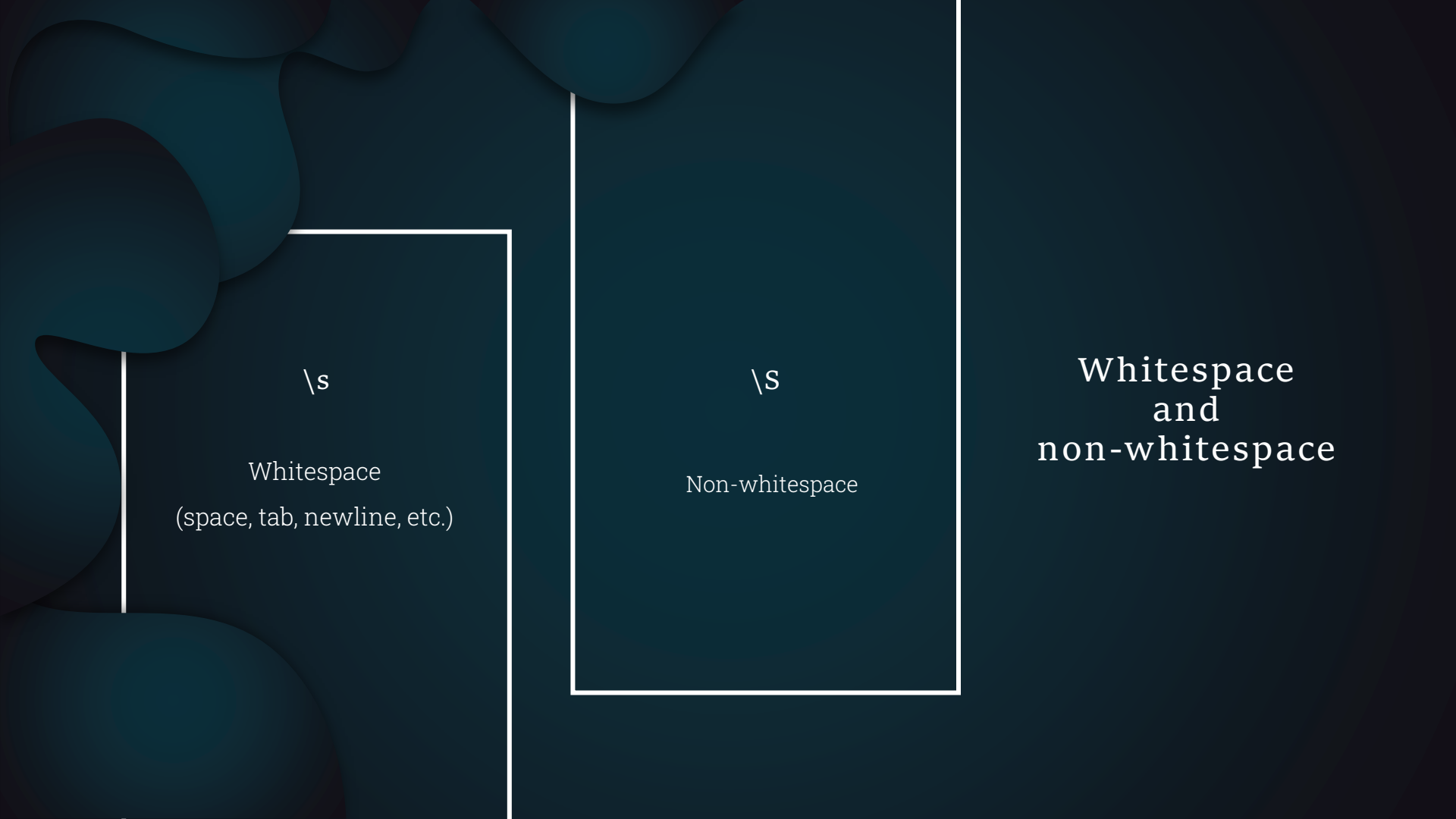
Same as `[0-9]`

`\D`

Non-Digit

Same as `[^0-9]`

Digits and
Non-Digits



The diagram is set against a dark teal background with abstract, layered organic shapes on the left. It features two white-outlined rectangular boxes. The left box contains the text '\s' at the top, followed by 'Whitespace' and '(space, tab, newline, etc.)' below it. The right box contains the text '\S' at the top, followed by 'Non-whitespace' below it. To the right of these boxes, the text 'Whitespace and non-whitespace' is displayed in a serif font.

\s

Whitespace
(space, tab, newline, etc.)

\S

Non-whitespace

Whitespace
and
non-whitespace

`\w`

word character: letters,
digits, underscore
([a-zA-Z0-9_])

`\W`

non-word character

Alphanumeric
and
Non-Alphanumeric

Quoting Special Characters

- Escape with backslash: `\.`, `\*`, `\+`, `\?`, `\(`, `\)`, `\[`, `\]`, `\{`, `\}`, `\\`, `\^`, `\$`, `\|`
- Example: To match "3.5" literally, use `3\.` (not `3.5` which matches `"3x5"` too)
- In some regex engines, `\Q` and `\E` quote a block

Greedy vs Lazy

01 Regex is greedy.

02 Greedy `.*` matches as much as possible

03 Lazy `.*?` matches as little as possible

04 Example:

String: "this is a thistles"

Greedy `th.*s` → "this is a thistles"

Lazy `th.*?s` → "this"

Practical Tool - gedit

- Enable regex search
 - (Ctrl+H, then check "Regular expression")
- Example find: `\bcat\b` (whole word)
- Replace with: `dog`

Practical Tool - vi/vim

- `:s/old/new/` replace first occurrence on current line
- `:s/old/new/g` replace all on current line
- `:%s/old/new/g` replace entire file
- With regex: `:%s/\d\{3\}/XXX/g` replace 3-digit numbers

REGEX IN JAVA

Now let's see how to use regex in
java including the syntax.

Pattern & matcher

- `find()` - finds next match
- `group()` - returns matched substring
- `start() / end()` - match positions
- `matches()` - requires full string match

```
1 import java.util.regex.Pattern;
2 import java.util.regex.Matcher;
3
4 String text = "The cat in the hat";
5 String regex = "cat";
6
7 Pattern pattern = Pattern.compile(regex);
8 Matcher matcher = pattern.matcher(text);
9
10 if (matcher.find()) {
11     System.out.println("Found at index: " + matcher.start());
12 }
```

Extract digits

Java - Character Classes, Quantifiers & Anchors

```
1 String text = "Order 123, price $45.67";
2 Pattern p = Pattern.compile("\\d+"); // Note double escape!
3 Matcher m = p.matcher(text);
4 while (m.find()) {
5     System.out.println(m.group()); // 123, 45, 67
6 }
```

Validate whole screen

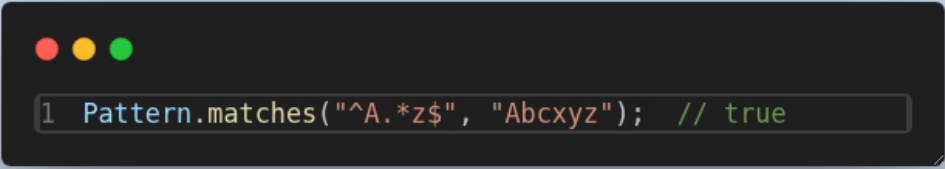
Java - Character Classes, Quantifiers & Anchors



```
1 String phone = "123-456-7890";  
2 boolean isValid = Pattern.matches("\\d{3}-\\d{3}-\\d{4}", phone);  
3 // true
```


Start/end anchors

Java - Character Classes, Quantifiers & Anchors



```
1 Pattern.matches("^A.*z$", "Abcxyz"); // true
```

Java – Capturing Groups & group()

Extracting parts of a match

```
1 String text = "user name is Nazanin and I am 20 years old";
2 Pattern p = Pattern.compile("user name is (\\w+) and I am (\\d+) years old");
3 Matcher m = p.matcher(text);
4
5 if (m.find()) {
6     System.out.println("Full match: " + m.group(0));
7     // "user name is Nazanin and I am 20 years old"
8     System.out.println("Name: " + m.group(1));    // "Nazanin"
9     System.out.println("Age: " + m.group(2));    // "20"
10 }
```

Java – Capturing Groups & group()

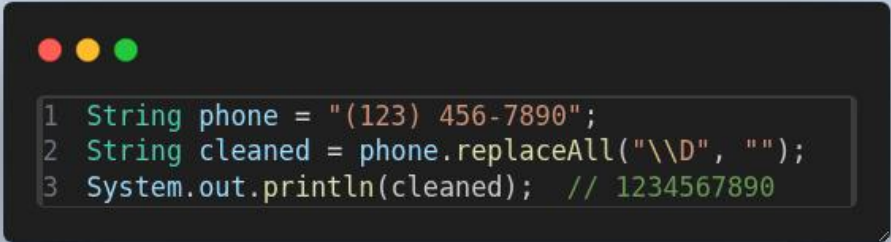
Multiple groups



```
1 String data = "John:25, Sarah:30";
2 Pattern p = Pattern.compile("(\\w+):(\\d+)");
3 Matcher m = p.matcher(data);
4
5 while (m.find()) {
6     System.out.println("Name: " + m.group(1) + ", Age: " + m.group(2));
7 }
8 // Name: John, Age: 25
9 // Name: Sarah, Age: 30
```

Java - Replacement (replaceAll, replaceFirst, Matcher.replaceAll)

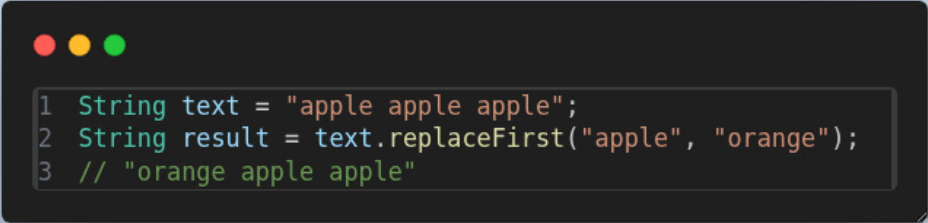
Basic replacement



```
1 String phone = "(123) 456-7890";  
2 String cleaned = phone.replaceAll("\\D", "");  
3 System.out.println(cleaned); // 1234567890
```

Java - Replacement (replaceAll, replaceFirst, Matcher.replaceAll)

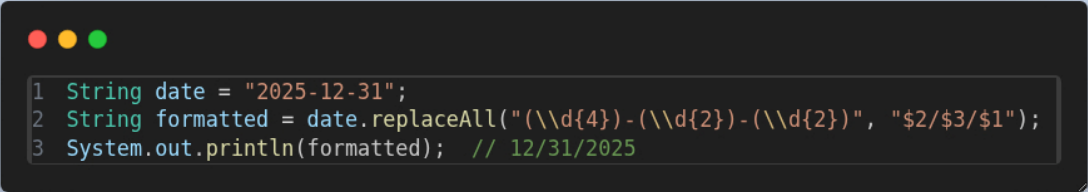
Replace first occurrence only



```
1 String text = "apple apple apple";  
2 String result = text.replaceFirst("apple", "orange");  
3 // "orange apple apple"
```

Java - Replacement (replaceAll, replaceFirst, Matcher.replaceAll)

Using backreferences in
replacement (\$1, \$2)



```
1 String date = "2025-12-31";  
2 String formatted = date.replaceAll("(\\d{4})-(\\d{2})-(\\d{2})", "$2/$3/$1");  
3 System.out.println(formatted); // 12/31/2025
```

Java - Replacement (replaceAll, replaceFirst, Matcher.replaceAll)

Using Matcher for
conditional replacement

```
1 Pattern p = Pattern.compile("\\d+");
2 Matcher m = p.matcher("Item 42 costs $99");
3 StringBuffer sb = new StringBuffer();
4
5 while (m.find()) {
6     int num = Integer.parseInt(m.group());
7     m.appendReplacement(sb, String.valueOf(num * 2));
8 }
9 m.appendTail(sb);
10 System.out.println(sb); // Item 84 costs $198
```



Thanks for your
attention